Лабораторная работа 5

СИСТЕМНЫЕ ВЫЗОВЫ

Цель лабораторной работы – изучение механизма внутренних прерываний и системных вызовов, модификация обработчика системных вызовов и внедрение набора системных вызовов в учебную операционную систему Pintos.

Основная рабочая директория: userprog/syscall.c

Теоретические сведения

Системные вызовы обеспечивают взаимосвязь между ядром и программами пользователя**.** Их можно рассматривать как функции, которые вызываются из пользовательских программ и выполняются ядром. Базовая реализация системных вызовов в ОС Pintos размещает аргументы с соответствующим номером системного вызова в регистры и вызывает прерывание 0x30. Это прерывание заставляет процессор остановить выполнение текущего задания, переключиться из режима пользователя в режим ядра и перейти к обработчику прерываний syscall_handler, который определен в userprog/syscall.c.

Прерывания служат для оповещения ядра системы о наступлении некоторого события. Многие механизмы операционной системы так или иначе взаимодействуют с прерываниями. В ОС Pintos прерывания можно разделить на две большие группы:

- Внутренние прерывания, т.е. прерывания, вызванные событиями в самом процессоре. Такими событиями могут быть системные вызовы, ошибки обращения по недопустимым адресам или деления на ноль. Так как внутренние прерывания вызваны процессором, они являются синхронными или синхронизированными с инструкциями процессора. Функция intr_disable() не осуществляет отключение внутренних прерываний.
- Внешние прерывания, т.е. прерывания, источники которых расположены вне CPU. Такие прерывания поступают от периферийных устройств, таких как системный таймер, клавиатура, диски. Внешние прерывания являются асинхронными, их поступление никак не синхронизировано с выполнением команд. Обработка внешних прерываний может быть отключена функцией intr_disable().

Операционная система Pintos в равной мере осуществляет обработку прерываний обеих групп.

Обработчик прерываний работает напрямую с ядром системы: все взаимодействия с ядром с помощью системных вызовов осуществляются через него. В threads/interrupt.h и threads/interrupt.c расположены основные структуры и функции обработки прерываний. В функции intr_init(), определенной в threads/interrupt.c, происходит установка таблицы векторов прерываний и связывание каждой точки входа с функцией intrNN_stub() (определена в threads/intr-stubs.S), где NN соответствует шестнадцатеричному номеру прерывания.

Основная работа функции intr_handler() заключается в вызове функции, предназначенной для обработки конкретного прерывания. Если такой функции не существует, intr_handler() завершается ошибкой и выводит информацию на консоль. Когда intr_handler() возвращает значение, ассемблерные функции в threads/intr-stubs.S восстанавливают ранее сохраненное состояние всех регистров процессора и передают управление процессору. В табл. 5.1. приведены основные структуры данных для работы с прерываниями.

Таблица 5.1.Работа с прерываниями

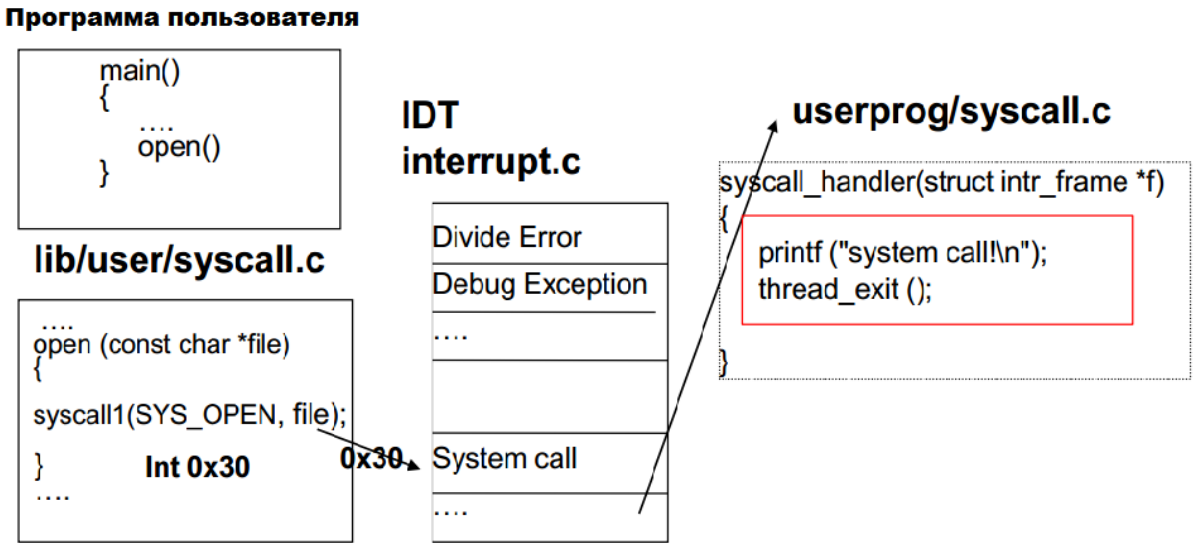
Объект:	Описание:
void intr_handler_func (struct intr_frame *frame)	Любая функция обработки прерываний должна быть объявлена подобным образом. Аргумент frame позволяет определить причину прерывания, а также состояние процесса, работа которого была прервана.
struct intr_frame	Состояние стека, сохраненное процессором и функцией intr_entry(). Содержит поля, в которые сохраняются значения регистров прерванного процесса, вектор прерывания, адрес следующей инструкции, указатель вершины стека прерванного процесса.

Обработчик внутренних прерываний имеет возможность модифицировать структуру intr_frame. Когда обработка прерывания завершается, изменения в intr_frame становятся изменениями в текущем состоянии процесса. Например, обработчик системных вызовов возвращает значение пользовательской программе, изменяя содержимое регистра eax. Базовая реализация обработчика прерываний выполняет вывод отладочного сообщения "system call!" и немедленно завершает вызвавшую обработчик программу:

```
static void syscall_handler (struct intr_frame *f UNUSED)
{
  printf ("system call!\n");
  thread_exit ();
}
```

На рис. 5.1. представлена текущая схема обработки системных вызовов.

Рисунок 5.1. Механизм обработки системных вызовов



Обратите внимание, что несколько файлов в исходных кодах Pintos имеют одинаковое название (src/lib/user/syscall.c иsrc/userprog/syscall.c), но выполняют совершенно разные задачи на разных уровнях.

Обработчики прерываний, как правило, должны работать с данными, предоставленными пользовательскими программами. Считывать такие данные коду ядра следует очень осторожно, т.к. код пользовательских программ мог передать указатель на нулевой адрес памяти, на не отображенный (не выделенный) участок памяти, а также на участок памяти, принадлежащий

виртуальному адресному пространству ядра (старше PHYS_BASE). Любые виды некорректных указателей должны быть отклонены без вреда как для самого ядра ОС, так и всех других выполняющихся пользовательских программ. Отклонение такой попытки предоставления некорректного указателя может быть реализовано, например, путем немедленного прекращения исполнения вызвавшего процесса.

Существует два способа сделать это правильно. Первый способ заключается в проверке достоверности предоставленного пользователем указателя перед любыми попытками разыменования (обращения по адресу, хранящемуся в нем). Если выбран этот способ, то следует рассмотреть функции в userprog/pagedir.c" и threads/vaddr.h. Так, функция is_user_vaddr проверяет принадлежит ли указанный адрес адресному пространству пользовательских программ, pagedir_get_page — проверяет выделена ли страница под указанный виртуальный адрес пользовательской программы и возвращает ее. Если страница для виртуального адреса p, переданного пользователем, выделена, то данные с адреса pg_round_up(p) по pg_round_down(p) могут быть безопасно считаны ядром.

Второй способ состоит в проверке только на то, что указатель пользователя ниже PHYS_BASE. Затем производится его разыменование. Если указатель — недопустимый, то обращение по такой памяти вызовет ошибку страницы "page fault". Соответствующую реализацию обработки ошибки следует внести в код функции page_fault () в "userprog/exception.c". Этот метод обычно выполняется быстрее, так как он использует MMU процессора, так что он, как правило, используется в реальных ядрах операционных систем (включая Linux и Windows).

В любом случае, вы должны убедиться, что не "утечек" ресурсов. Например, предположим, что система выделила память с помощью malloc и захватила некоторый "замок". Если вы столкнулись с недопустимым указателем пользователя после этого, то вы все равно должны быть уверены, что "замок" будет разблокирован, а память — освобождена. В случае применения первой техники реализация такой задачи выглядит довольно простой задачей. Труднее справиться, если недействительный указатель вызывает ошибку страницы, потому что нет простого способа, чтобы вернуть код ошибки при нарушении доступа к памяти.

Порядок выполнения работы

1. Модифицировать текущий механизм обработки системных вызовов. Обработчик системных вызовов в userprog/syscall.c должен читать номер и аргументы системного вызова из стека и при завершении обработки возвращать значение пользовательской программе, изменяя содержимое регистра eax (необходимо будет использовать указатель стека esp из структуры intr_frame). Схема возможного механизма представлена на рис. 5.2.

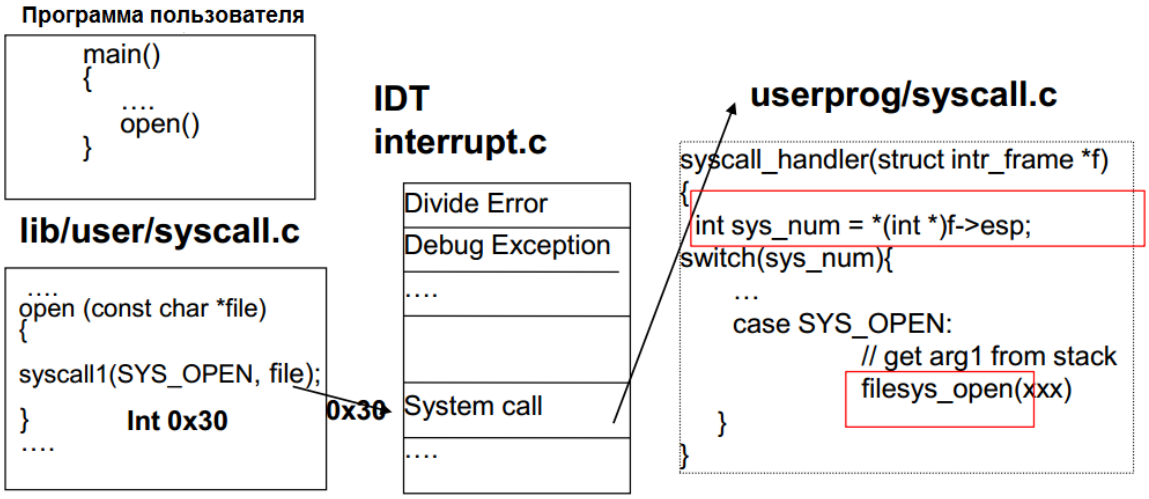
Внести соответствующие изменения в функцию syscall_handler(). Номер системного вызова и его аргументы передаются пользовательской программой через память, адрес массива сохранен в регистре esp. Коду ядра ОС следует проверить, что адрес, переданный пользовательской программой в esp — действительный, т.е. указывает на выделенную память.

Если представить данные, расположенные по адресу, на который указывает esp как массив чисел, то первый элемент этого массива является номером системного вызова, второй элемент — первым аргументов вызванной системной функции, третий — вторым аргументом и т.д.

```
static void syscall_handler (struct intr_frame *f)
{
    int nr; int* args;
    validate_user_pointer((const void*) f->esp); // Проверка действительности указателя
                                                 // (реализуйте такую или подобную функцию)

    nr = *(int*) f->esp; // Номер системного вызова
    args = (int*) f->esp + 1; // Аргументы системного вызова, args[0] – первый, args[1] – второй, ..
}
```

Рисунок 5.2. Модификация механизма обработки системных вызовов



2. Реализовать набор системных вызовов, представленный в табл. 5.2. Для внедрения системных вызовов в первую очередь необходимо продумать способы, которыми будут осуществляться чтение и запись в виртуальное пространство пользователя (номер системного вызова и его аргументы находятся в пользовательском стеке). Номера системных вызовов (константы SYS_ZZZ) определены в lib/syscall-nr.h. Каркасы функций, выполняющие обращение к ядру через "шлюз" прерывания 0x30, расположены в lib/user/syscall.c. Для разбора аргументов командной строки может быть использован код из предыдущей л/р.

Таблица 5.2. Системные вызовы ОС Pintos

Прототип системного вызова:	**Назначение:**	**Тесты:**
void halt (void)	Завершает работу ОС Pintos. Используйте функцию power_off(), которая определена в threads/init.h.	halt
void exit (int status)	Завершает работу пользовательской программы, возвращая status ядру. Статус 0 означает безошибочное выполнение программы.	exit
pid_t exec (const char *cmd_line)	Запускает исполняемый файл, имя которого задано в cmd_line (с учетом всех аргументов) и возвращает уникальный идентификатор нового пользовательского процесса, который создается в результате запуска исполняемого файла. Системный вызов должен возвращать значение -1, если программа не может быть запущена (например, исполняемый файл не найден на диске). Реализация обработчика этого системного вызова возможна через вызов process_execute. В функции следует исправить некорректную реализацию: в случае, если запускаемый файл не будет найден на диске, process_execute возвращается без кода ошибки. Следует синхронизировать load и process_execute таким образом, чтобы в случае отсутствия файла на диске, process_execute вернула бы ошибку. Следует учитывать, что входным аргументом является	exec-once exec-multiple exec-arg exec-missing exec-bad-ptr

	массив символов, передаваемый пользовательской программой по указателю. Корректность указателя должна быть проверена ядром перед разыменовыванием.	
int wait (pid_t pid)	Ожидает, пока дочерний процесс с идентификатором pid не завершит выполнение, затем возвращает код (статус) завершения дочернего процесса. Если дочерний процесс не вызвал функцию exit(), но был завершен ядром, то системный вызов wait(pid) должен возвращать значение -1. Системный вызов также должен возвращать -1 в следующих случаях: • pid не является дочерним процессом выполняющего системный вызов wait процесса; • системный вызов wait над pid уже был выполнен; Процесс может порождать несколько дочерних процессов, выполнять системный вызов wait для них в любом порядке и даже завершать свое выполнение, не дождавшись завершения своих дочерних процессов. Перед тем, как приступить к внедрению системного вызова, нужно реализовать функцию process_wait() в userprog/process.c, и на ее основе выполнять системный вызов. Обратите внимание, что в отличие от предыдущей работы, process_wait обязательно должна проверять — является ли процесс, указанный в ее аргументе дочерним от текущего процесса.	wait-simple wait-twice wait-killed wait-bad-pid
bool create (const char *file, unsigned initial_size)	Создает новый файл с именем file и размером initial_size. Возвращает “true”, если файл успешно создан, и “false” в противном случае. Реализовать обработку системного этого вызова можно с помощью функций, входящих в состав драйвера файловой системы pintos. Заголовочным файлом драйвера является файл filesys/filesys.h Следует учитывать, что входным аргументом является массив символов, передаваемый пользовательской программой по указателю. Корректность указателя должна быть проверена ядром перед разыменовыванием.	create-empty create-long create-normal create-exists create-null create-bad-ptr create-bound
bool remove (const char *file);	Удаляет файл с указанным именем с диска. Если файл не существует — возвращает false.	
int open (const char *file)	Открывает файл с именем file. Возвращает целочисленное значение дескриптора файла fd или -1, если файл не может быть открыт. Дескрипторы 0 и 1 зарезервированы для стандартных потоков ввода-вывода: fd 0 — STDIN_FILENO, fd 1 — STDOUT_FILENO. Функция может быть реализована с использованием драйвера файловой системы (filesys_open). Следует учесть, что процесс может открывать и закрывать значительное количество файлов во время работы. Информацию обо всех открытых файлах для каждого процесса следует хранить в какой-либо вашей структуре	open-missing open-normal open-twice open-boundary open-empty open-null open-bad-ptr

	данных (хранится соответствие fd и struct file*).	
void close (int fd)	Закрывает файл с дескриптором fd. Если программа пытается закрыть выходной поток (fd=1) то, ядро должно завершить ее с ошибкой (код выхода -1). При своем завершении процесс неявно закрывает все открытые файловые дескрипторы, словно вызывает функцию close для каждого из них. Это поведение обязательно должно быть реализовано в ядре. Следует заметить также, что каждому открытому файлу соответствует некоторая структура struct file*. Для каждой из таких структур должна быть вызвана file_close.	close-normal close-twice close-stdin close-stdout close-bad-fd
int filesize (int fd)	Возвращает размер открытого файла в байтах. Обработчик может быть реализован через функцию file_length драйвера файловой системы pintos. Поддержка системного вызова требуется для работы тестов семейства read и write.	
int read (int fd, void *buffer, unsigned size)	Читает size байт из файла с дескриптором fd в буфер. Возвращает число считанных байт или -1, если файл не может быть прочитан. fd=0 выполняет чтение с клавиатуры с помощью функции input_getc(), определенной в devices/input.h. Чтение данных файла можно реализовать через функцию file_read драйвера файловой системы.	read-normal read-zero read-bad-ptr read-boundary read-stdout read-bad-fd
int write (int fd, const void *buffer, unsigned size)	Записывает size байт в файл с дескриптором fd. Возвращает число записанных байт или -1, если запись в файл невозможна. Запись в файл после EOF должна увеличивать размер файла, но в текущей реализации файловой системы Pintos такой механизм не предусмотрен. Система ожидает от пользователя запись в пределах заданного при создании размера файла. Если fd = STDOUT_FILENO (значение 1), то система должна выполнять вывод на консоль. Используйте функцию putbuf(), определенную в lib/kernel/stdio.h и lib/kernel/console.c. Запись данных можно реализовать через функцию file_write драйвера файловой системы.	write-normal write-zero write-bad-ptr write-boundary write-stdin write-bad-fd

3. Проанализировать ситуации, предлагаемые встроенными тестами ОС Pintos по каждому из системных вызовов. Описать и привести примеры из кода, каким образом в реализованных системных вызовах были решены те или иные проблемы, например, как была выполнена защита от двойного обращения к файлу системным вызовом close() (тест close-twice). Составить сводную таблицу по всем тестам (в качестве образца можно взять табл. 5.3). Полный список тестов по данной лабораторной работе приведен в табл. 5.2.

Таблица 5.3. Пример оформления результатов анализа тестов

Название теста:	**Рассматриваемая проблема, описание теста:**	**Как была решена проблема:**
create-exists	Тестовая программа создает набор файлов, а затем пытается осуществить повторное создание файлов с теми же именами.	В моей реализации системного вызова create осуществляется проверка наличия одноименного файла, поэтому создание двух файлов с одинаковыми именами представляется невозможным.
read-zero	…	…
open-missing	…	…
…	…	…

Содержание отчета

1. Блок-схема реализованного механизма обработки системных вызовов.
2. Исходные коды с внесенными модификациями и комментариями.
3. Таблица результатов анализа тестовых ситуаций.
4. Выводы тестов.
5. Ответы на контрольные вопросы.
6. Результаты выполнения индивидуального задания: описание реализации программы, и организации хранения данных, исходные тексты, примеры входных и выходных данных на тестовых прогонах.